



µLab worksheet · a cow of your own

Group _____ Date _____ Time _____

Driver (writes) _____ Mapper (does not write) _____

Hand this to the teacher when the round ends.

1 · WHOSE ACCOUNT IS THIS ON?

The driver's GitHub handle, **one character per box**. Everything you publish today lands under this name, and this sheet is the only thing that connects it back to your group. If we cannot read it, we cannot find your work.

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

The repository you create in step 1 _____

2 · THE FIRST HALF, IN FOUR STEPS

1 ☐ Take the template

Open `github.com/oesteban/whalesay` and press **Use this template** → **Create a new repository**, under **your own** account.

2 ☐ Branch, and add your cow

`git checkout -b group-N · write group-N.cow · commit · push.`

3 ☐ Build it and make it talk

`docker build -t my_whale . then docker run --rm my_whale -f group-N "hi from group N"`

4 ☐ Open the pull request

Against `main`, **in your own repository**. Then watch the **Actions** tab.

3 · WHAT YOU BUILT

After step 3, run `docker images`.

Size of `my_whale` _____ Size of `alpine` _____

You added one small text file to `alpine` and got a much larger image. What else went in, and which line of the `Dockerfile` put it there?

Add the two sizes together. Now check how much disk Docker is really using with `docker system df`. The numbers disagree — why?

4 • WHAT `--rm` WAS FOR

Run the same cow twice, then look at what is left behind.

```
docker run --rm my_whale -f group-N "one"
docker run my_whale -f group-N "two" < NO --rm
```

docker ps shows _____ docker ps -a shows _____

Both printed a cow and both finished. One of them is still on your machine. What is still there, and what is it still holding?

Run the no- `--rm` version three more times. How many entries in `docker ps -a` now? _____ Clear them, and write the command you used.

5 • A CONTAINER YOU CAN WALK BACK INTO

`docker run -d` detaches. Try it on `my_whale` as it is, first, and see what `docker ps` says a few seconds later.

Now give it something that does not finish immediately:

```
docker run -d --name cow --entrypoint sleep my_whale 600
docker exec cow ls /usr/local/share/cows/
docker exec cow cowsay -f group-N "from inside"
```

Why did the first attempt need no `--entrypoint` and this one does?

Compare `docker exec cow whoami` with `whoami` in your own terminal, and `docker exec cow ls /` with `ls /`. Same machine or not? Say how you know.

Stop it and remove it. Commands used: _____

6 • WHAT THE IMAGE REMEMBERS

Run `docker image history my_whale`.

Lines your Dockerfile added _____ Largest single line _____

Most lines are 0B. What does a line of 0B mean actually happened?

Add a second cow file and rebuild with the same command. Watch the output.

Which steps re-ran, and which said `CACHED`? Write the first step that re-ran. _____

You changed one file. Why did that step, and not an earlier one, have to run again? Guess, and say what would test your guess.

7 • SOMETHING ELSE BUILT IT

After step 4, open the **Actions** tab on your repository.

The run took _____ It ran on _____

That machine has never seen your laptop. What did it need from you to build your image, and where did it get it?

The run finished green, and nothing was published. Say where you looked to be sure of that.

8 • A WEB SERVER THAT STARTS CONTAINERS

A web server with no cowsay in it. Every request it receives, it runs `docker run` and a fresh cow container prints the answer and dies. Two files, in a `web/` directory:

```
web/Dockerfile
FROM alpine
RUN apk add --no-cache busybox-extras docker-cli
COPY cgi-bin/ /www/cgi-bin/
ENTRYPOINT ["httpd", "-f", "-p", "80", "-h", "/www"]

web/cgi-bin/cow — and it must be executable

#!/bin/sh
echo "Content-type: text/html"
echo
say=$(printf '%s' "$QUERY_STRING" | sed -n 's/^.*message=([^\&]*)\.*$/\1/p' | sed 's/+ /g')
[ -z "$say" ] && say="try /?message=hello"
echo "<pre>"
docker run --rm my_whale -f group-N "$say"
echo "</pre>"
```

```
chmod +x web/cgi-bin/cow
docker build -t cow-web ./web
docker run --rm -p 8080:80 -v /var/run/docker.sock:/var/run/docker.sock cow-web
```

```
curl "localhost:8080/cgi-bin/cow?message=hello+302"
```

Before you run it: there is no `cowsay` anywhere in the `web` image. So where does the cow come from?

Open a second terminal, run `docker events`, and load the page twice. Write the sequence of words it prints for one request.

How many cow containers exist on your machine between two requests? _____ Which flag in the script decided that?

That last command handed the container `/var/run/docker.sock`. Name one thing this web server could now do to your laptop that has nothing to do with cows.

9 • FOUR LINES, FOUR LAYERS

Go back to the `web/Dockerfile` in box 8 and take its four instructions one at a time. In your own words: what did each one put in the image?

`FROM alpine`

What is in the image after this line and before the next one runs, and where did it come from? Then: why `alpine` rather than, say, `ubuntu`? Box 3 measured your evidence.

`RUN apk add --no-cache busybox-extras docker-cli`

One of these two packages is the web server, the other is the `docker` command. Which is which — and is the Docker **daemon** in this image?

`COPY cgi-bin/ /www/cgi-bin/`

Where does `cgi-bin/` live before this line runs, and where does it live afterwards? Name both places precisely enough that someone could go and look.

`ENTRYPOINT ["httpd", "-f", "-p", "80", "-h", "/www"]`

Three flags: `-f`, `-p 80`, `-h /www`. Which one keeps the container alive, and what did box 5 have to do for a command that lacked it? And `80` turns up again in the `docker run` line as `8080:80` — say which of those two it is.

10 • HIDING CONTENTS FROM THE BUILD CONTEXT

At the root of your repository, next to the `Dockerfile` and **not** inside `web/`, create a file called `.dockerignore` holding one line:

`web/`

Two small files make a difference nobody can see, so put something heavy in there first, then build the cow image twice: once as it is, once with the `web/` line commented out (`#` starts a comment).

```
dd if=/dev/urandom of=web/junk bs=1024 count=20000
docker build -t my_whale . ← read the line about the build context
```

Read that first command before you run it, because you are about to quote its result as a measurement. `dd` copies blocks: `bs` is the size of one block in bytes, `count` is how many of them.

So `web/junk` will be _____ and `ls -l` says _____

And what is in it? `/dev/urandom` is not a file sitting on your disk. Say what it is and what you therefore just wrote into `web/`.

Context with `web/` ignored _____ and without _____

Your `Dockerfile` copies `cowsay`, `*.cow` and `docker.cow` by name, so nothing in `web/` was ever going to end up inside `my_whale`. What did carrying it cost you, then, and who was it being carried to?

Put the line back and delete `web/junk`, or the frontend build would carry it. Which is the question: you just told Docker to ignore `web/`, so why does `docker build -t cow-web ./web` still work at all?

11 · EXTRA MILE · LET THE REQUEST PICK THE COW

Same server, one more parameter: the cow comes from the URL instead of being fixed in the script.

```
curl "localhost:8080/cgi-bin/cow?message=hello+302&cow=group-N"
```

One line of the script already does this for `message`. Write the line you added for `cow`, and what it falls back to when nobody asks for one.

Ask for a cow the image does not have (`&cow=nonexistent`). What does the page show, and which of the two containers produced that text?

Both values now come from the request, but only one of them can make `docker run` fail. Which one, and what does that say about the difference between them?

Done when `?message=hello` makes your cow say hello, and `docker events` shows a container created and destroyed on every request.